

JAVA INTERVIEW PREPARATION

CHAPTER 12

SEALED CLASSES AND CONTROLLED HIERARCHIES

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Sealed Classes and Controlled Hierarchies: Modeling Closed Alternatives

Senior / Lead Java Developer Reading Chapter

Sealed classes and interfaces became permanent in Java 17. They allow a type to declare which classes may directly extend or implement it. This turns an inheritance hierarchy from an open invitation into a controlled set of alternatives.

The feature is not mainly about preventing other developers from subclassing. Its deeper value is modeling: some domains contain a finite set of meaningful cases. A payment outcome is approved, declined, or pending. A command is created, cancelled, or fulfilled. An expression is a constant, addition, or multiplication. When the compiler knows the permitted cases, code can reason exhaustively about them.

Senior design must still decide whether a hierarchy should be closed, how it may evolve, where permitted classes live, and whether behavior belongs on the variants or in pattern-matching consumers.

Open and Closed Type Hierarchies

An ordinary interface is open:

```
interface PaymentResult {}
```

Any accessible class can implement it. A consumer cannot assume that today's known implementations are the complete domain.

A sealed interface closes direct extension:

```
sealed interface PaymentResult
    permits Approved, Declined, Pending {}

record Approved(TransactionId id) implements PaymentResult {}
record Declined(DeclineReason reason) implements PaymentResult {}
record Pending(ReviewId reviewId) implements PaymentResult {}
```

```
PaymentResult (sealed)
|
+-- Approved
+-- Declined
+-- Pending

no other direct implementation is permitted
```

This lets the type itself document the complete top-level set. It also enables exhaustive pattern matching without a catch-all default when every permitted case is covered.

Declaring the Permitted Subtypes

The `permits` clause names direct subtypes:

```
public sealed interface DeliveryStatus
    permits InTransit, Delivered, FailedDelivery {}
```

The compiler can infer the permitted subtypes when they are declared in the same source file, so `permits` may be omitted:

```
sealed interface DeliveryStatus {}

record InTransit(Instant expectedArrival) implements DeliveryStatus {}
record Delivered(Instant deliveredAt) implements DeliveryStatus {}
record FailedDelivery(String reason) implements DeliveryStatus {}
```

In a named module, permitted direct subtypes must be in the same module. In the unnamed module, they must be in the same package. These locality rules ensure that the compiler and runtime can identify the controlled family.

Sealing governs direct subtypes only. Each permitted subtype must declare how the hierarchy continues.

Every Permitted Subtype Chooses a Policy

A permitted class must be one of:

- **final**: the branch ends.
- **sealed**: the branch remains controlled by another permits list.
- **non-sealed**: the branch becomes open to further subclassing.

```
sealed abstract class Account
    permits RetailAccount, CorporateAccount, LegacyAccount {}

final class RetailAccount extends Account {}

sealed class CorporateAccount extends Account
    permits DomesticCorporateAccount, InternationalCorporateAccount {}

non-sealed class LegacyAccount extends Account {}
```

```
Account (sealed)
|
+-- RetailAccount (final)      branch closed
|
+-- CorporateAccount (sealed)  controlled sub-branches
|   +-- Domestic...
|   +-- International...
|
+-- LegacyAccount (non-sealed) branch reopened
    +-- arbitrary subclasses
```

non-sealed is a deliberate escape hatch. Once a branch is reopened, consumers cannot enumerate all concrete descendants of that branch. The compiler can still treat the direct `LegacyAccount` branch as covered, but code inside it may need polymorphism for unknown subclasses.

Sealed Does Not Mean Immutable or Final

A sealed class controls who may extend it. It says nothing about field mutability, thread safety, equality, or lifecycle.

```
sealed class Job permits QueuedJob, RunningJob {
    private Instant lastUpdated;

    void touch(Instant now) {
        lastUpdated = now;
    }
}
```

Records used as permitted variants are final and have fixed component references, making them convenient for immutable case data. But sealing and immutability solve different problems:

```
sealing      -> who can extend the type?
immutability -> can observable state change?
final class  -> can this particular class be extended?
encapsulation -> who can access or change representation?
```

Do not describe a sealed hierarchy as automatically immutable.

Algebraic Data Type Thinking

A sealed hierarchy models a sum of alternatives. Each record variant models a product of its component values.

```
sealed interface QuoteResult {
    record Quoted(Quote quote, Duration validity) implements QuoteResult {}
    record Unavailable(UnavailableReason reason) implements QuoteResult {}
    record ManualReview(CaseId caseId) implements QuoteResult {}
}
```

Conceptually:

```
QuoteResult = Quoted(Quote x Duration)
              | Unavailable(Reason)
              | ManualReview(CaseId)
```

This makes illegal combinations harder to construct. A single class with nullable `quote`, `reason`, and `caseId` could represent contradictory or empty states. Separate variants ensure each case carries exactly the data it requires.

Exhaustive Pattern Matching

Java 21 pattern matching for switch works naturally with sealed types:

```
String message(QuoteResult result) {
    return switch (result) {
        case QuoteResult.Quoted quoted ->
            "Valid for " + quoted.validity();
        case QuoteResult.Unavailable unavailable ->
            "Unavailable: " + unavailable.reason();
        case QuoteResult.ManualReview review ->
            "Review case " + review.caseId();
    };
}
```

No `default` is required because all permitted cases are handled. If a new permitted subtype is added, exhaustive switches fail to compile until updated. That failure is useful: it identifies every place that must decide what the new domain case means.

Avoid adding a default solely to silence exhaustiveness:

```
default -> "Unknown";
```

A default can hide new variants from the compiler's assistance. Use it only when forward-compatible fallback behavior is genuinely part of the contract.

Behavior on Variants versus External Switching

Classic object orientation places behavior on each subtype:

```
sealed interface PricingRule permits FixedPrice, PercentageDiscount {
    Money apply(Money original);
}

record FixedPrice(Money price) implements PricingRule {
    public Money apply(Money original) {
        return price;
    }
}
```

Data-oriented design may keep variants as data and place operations in processors using exhaustive switch.

Neither is universally superior. Use polymorphic methods when behavior belongs intrinsically to each variant and new operations are uncommon. Use external exhaustive processing when the set of variants is stable but many independent operations, serializers, validators, or projections act on them.

Favor polymorphism	Favor exhaustive processing
-----	-----
variants change frequently	operations change frequently
behavior owned by each case	case data is transparent
open extension is useful	closed alternatives are central
callers need one operation	many consumers interpret cases

Sealed hierarchies allow both styles. Do not turn every method into a switch merely because pattern matching is available.

Domain Results Instead of Exceptions or Optional

Optional expresses present or absent but cannot carry multiple business outcomes. Exceptions represent exceptional failure but are awkward for routine alternatives.

```
sealed interface PlaceOrderResult {
    record Accepted(OrderId orderId) implements PlaceOrderResult {}
    record Rejected(List<ValidationError> errors) implements PlaceOrderResult {
        Rejected {
            errors = List.copyOf(errors);
        }
    }
    record Duplicate(OrderId existingOrderId) implements PlaceOrderResult {}
}
```

The caller must handle each normal business result. Infrastructure failure can still propagate separately as an exception.

normal business alternatives	-> sealed result
expected simple absence	-> Optional
operational/system failure	-> exception or failure channel

This prevents "empty" or generic exceptions from collapsing meaningful states.

Designing Hierarchy Boundaries

A hierarchy should be sealed only when the owner can responsibly enumerate and evolve its cases.

Good candidates include:

- Internal workflow states.
- Compiler or rules-engine syntax trees.
- Command and event families controlled by one service.
- Domain outcomes with a finite vocabulary.

- Configuration variants owned by one module.

Poor candidates include public service-provider interfaces intended for third-party plugins, framework extension points, and open strategy APIs where unknown implementations are a feature.

```
// Open by design: external modules provide exporters.  
public interface ReportExporter {  
    void export(Report report, OutputStream output);  
}
```

Sealing this interface would prevent ecosystem extension or force every provider into the owning module.

Packages, Modules, and Accessibility

The hierarchy's physical organization matters. In a named module, permitted types may live in different packages within that module. In the unnamed module, they must share a package.

Accessibility rules still apply. A public sealed interface can permit package-private implementations, but external callers then cannot name those implementations directly. This can be useful when clients should call operations on the parent rather than construct cases themselves, but it complicates exhaustive switching outside the package.

If callers are expected to pattern-match variants, make the necessary types and component accessors accessible. If not, expose parent-level behavior and keep variants encapsulated. Do not mix these models accidentally.

Sealed Types and Framework Proxies

Frameworks may create runtime subclasses for lazy loading, transactions, mocking, or interception. A sealed or final class rejects subclasses not in its permits structure.

This matters for:

- ORM entity proxies.
- Method-interception frameworks.
- Runtime code generation.
- Mocking tools that subclass concrete classes.

Interfaces are often proxied through interface implementations, but a sealed interface also restricts which implementation classes are legal. Verify the framework's proxy mechanism before sealing types used at integration boundaries.

Do not weaken a meaningful domain hierarchy merely for a test mocking style. Prefer testing through interfaces, fakes, composition, or actual value instances. But at infrastructure boundaries, proxy compatibility can be a legitimate design constraint.

Evolution and Compatibility

Adding a permitted subtype is source-incompatible for consumers with exhaustive switches: they must add a case. This is often exactly the desired compile-time pressure inside one codebase.

Across independently deployed libraries, however, consumers may not recompile immediately. A newer producer can send a variant an older consumer cannot deserialize or interpret. Sealed types do not solve wire compatibility.

For events and APIs:

- Version schemas deliberately.

- Decide whether unknown variants are rejected, ignored, or mapped to a fallback.
- Avoid exposing Java class names as protocol discriminators.
- Test old readers with new payloads and new readers with old payloads.
- Coordinate deployment when exhaustive understanding is required.

The closed Java type family and the externally versioned message family are related but distinct contracts.

Nested and Generic Sealed Hierarchies

Nesting variants inside the sealed interface keeps a small family together:

```
sealed interface LookupResult<T> {
    record Found<T>(T value) implements LookupResult<T> {}
    record Missing<T>(String key) implements LookupResult<T> {}
    record Failed<T>(String code) implements LookupResult<T> {}
}
```

Generic sealed types can model reusable result containers, but avoid erasing domain meaning behind one universal `Result<T>`. A payment decline, validation rejection, and database failure have different payloads, metrics, retry policies, and HTTP mappings.

Use a generic family when the alternatives are genuinely consistent across contexts. Use a domain-specific family when the names and data improve decisions.

Invariants and Defensive Copies

Sealed hierarchies prevent unknown direct variants; each permitted type must still enforce its own invariants.

```
record Rejected(List<ValidationError> errors)
    implements PlaceOrderResult {
    Rejected {
        errors = List.copyOf(errors);
        if (errors.isEmpty()) {
            throw new IllegalArgumentException("rejection requires errors");
        }
    }
}
```

Without validation, the finite set of cases can still contain invalid values. Model correctness is the combination of:

```
permitted cases
+
valid payload for each case
+
exhaustive consumer behavior
```

Testing Sealed Models

Test every permitted case and the policies that interpret them.

```
@ParameterizedTest
@MethodSource("quoteResults")
void everyQuoteResultProducesClientMessage(QuoteResult result) {
    assertFalse(message(result).isBlank());
}
```

More important tests include:

- Variant constructor invariants.
- Serialization discriminators and payloads.
- Exhaustive mapping to HTTP, events, persistence, and metrics.
- Framework proxy or reflection integration.
- Compatibility with older message versions.
- Behavior of non-sealed branches with unknown descendants.

Compiler exhaustiveness prevents an omitted source case after recompilation, but it does not prove the business behavior of each case is correct.

Production Perspective

Sealed-hierarchy incidents arise at evolution and integration boundaries. A new result variant compiles inside one service but an older event consumer cannot deserialize it. A default switch silently maps a newly introduced fraud-review case to generic failure. An ORM attempts to proxy a sealed entity. A non-sealed escape branch allows an implementation whose invariants the parent assumed impossible. A record variant contains a mutable List and changes after publication.

When reviewing or diagnosing a sealed family, ask:

- Is the domain actually closed, and who owns all cases?
- Does every branch deliberately choose final, sealed, or non-sealed?
- Are consumers using useful exhaustiveness or hiding changes behind default?
- Should behavior be polymorphic or externally matched?
- Can frameworks legally proxy or instantiate the types?
- Are variants exposed across independent deployment or serialization boundaries?
- How are unknown future wire variants handled?
- Does each variant validate and protect its payload?

Interview-Ready Summary

A sealed class or interface restricts its direct subtypes to a declared or inferred permitted set. Permitted direct classes must be final, sealed, or non-sealed, defining whether each branch ends, remains controlled, or reopens.

Sealing is different from immutability and finality. It controls extension, not state mutation. Sealed interfaces often combine with record variants to model a finite set of alternatives with transparent payloads.

Closed hierarchies enable exhaustive Java 21 pattern switches. Omitting an unnecessary default lets the compiler identify consumers that must handle a newly added variant. Adding a case is still a deliberate source and deployment compatibility event.

Use sealed results for multiple normal business outcomes, Optional for simple expected absence, and exceptions for operational failures. Choose polymorphic methods when behavior belongs to variants; choose external pattern processing when many independent operations interpret a stable family.

Do not seal plugin and framework extension points that are intentionally open. Consider module locality, accessibility, runtime proxies, serialization, schema evolution, and independently deployed consumers before making a public hierarchy closed.

Revision Notes

- Sealed classes and interfaces became permanent in Java 17.
- A sealed type controls its direct subtypes.
- The permits clause names allowed direct subtypes.
- Permits may be inferred for subtypes declared in the same source file.
- In named modules, permitted types must be in the same module.
- In the unnamed module, permitted types must be in the same package.
- Every permitted class must be final, sealed, or non-sealed.
- Final ends a branch; sealed controls it; non-sealed reopens it.
- Sealing does not imply immutability, thread safety, or value equality.
- A sealed hierarchy models a finite sum of alternatives.
- Record variants model the fixed payload of each alternative.
- Separate variants prevent contradictory nullable-field combinations.
- Java 21 pattern switches can exhaustively cover sealed families.
- Avoid default when compiler-enforced handling of new variants is valuable.
- Use a default only when fallback behavior is genuinely required.
- Polymorphism suits behavior owned by variants.
- External pattern matching suits many operations over a stable case family.
- Sealed results express multiple routine business outcomes.
- Optional is better for simple absence; exceptions remain for operational failure.
- Seal only domains whose owner can enumerate and evolve all cases.
- Keep plugin and provider extension points open when third-party implementations are intended.
- Accessibility affects whether external callers can name and match variants.
- Framework proxies may conflict with sealed or final types.
- Adding a variant intentionally breaks exhaustive source consumers after recompilation.
- Sealed Java types do not solve wire-version compatibility.
- Version event and API discriminators independently of Java class names.
- Generic sealed results are useful only when alternatives share real semantics.
- Each variant must still validate and defensively copy its payload.
- Compiler exhaustiveness does not prove business behavior is correct.
- Test cases, mappings, serialization, proxies, and compatibility.
- Choose the closed boundary from domain ownership, not syntax preference.